

New

Super Agent

Home

Skills

Case

Workflows

Drive

More

High-speed crypto data scraper and terminal archit... 2

Share

Role & Context

You are a Principal Software Architect and Quantitative Trading System Engineer specializing in high-speed, low-latency data pipelines, browser automation, and real-time execution engines.

We are building a production-grade, automated high-speed scraper and data-collection terminal dashboard. Below are the functional specifications, performance requirements, and execution steps.

📄 System Specifications

1. Browser Automation & Layout Setup

- **"Browser Instance"**: A single debugging Chrome instance (playwright or Puppeteer/CDP connection).

- **"Tabs"**: Exactly two Chrome tabs.

- **"URL"**: Both tabs open the same layout link: https://www.coinglass.com/tv/layout/s9.

- **"Layout Containers"**: Each tab displays a 9-chart grid layout (total 18 assets, 15-minute timeframe).

- **"Asset Allocation"**:

- **"Tab 1"**: ["BTCUSD", "ETHUSD", "XRPUSD", "SOLUSD", "BNBUSD", "DOGEUSD", "ADAUSD", "TRXUSD", "LINKUSD"]
- **"Tab 2"**: ["AVAXUSD", "SUIUSD", "NEARUSD", "DOTUSD", "LTCUSD", "XAUUSD", "XAGUSD", "CLUSD", "NATGASUSD"]

2. Indicator Injection & Processing

On each container, the system must inject the following 7 indicators. Use the most robust, high-speed injection method (e.g., direct JS console injection, interception, or TV API hooks):

1. **"Aggregated Futures Cumulative Volume Delta (CVD)"** (coinglass: "futures cumulative")

2. **"Aggregated Spot Cumulative Volume Delta (CVD)"** (coinglass: "spot cumulative")

3. **"Relative Strength / RSI"** (coinglass: "rsi") (tv: "rsi")

4. **"Funding Rates"** (coinglass: "funding rate")

5. **"Aggregated Liquidations"** (coinglass: "liquidations")

6. **"Long/Short Ratio (Accounts)"** (coinglass: "long/short")

7. **"Aggregated Open Interest (STABLECOIN-margined, Candles)"** (coinglass: "open interest")

3. Real-Time Scraper & Dashboard

- **"Mechanism"**: Perform DOM analysis or network interception/JS mapping on both tabs.

- **"Refresh Rate"**: 0.5 seconds.

- **"Display"**: Output the real-time data in the terminal in a clean, tabular format.

4. Historical Seeding Protocol (Excel Export)

For each asset, the system must perform a backward seeding routine:

- Position the cursor on the latest candle.

- Press 'ALT+R' to reset the chart visual.

- Hit the left arrow key sequentially to scroll backward, scraping and collecting data for exactly "50 candles".

- Dump all 50 candles' OHLCV + indicators data to a local Excel spreadsheet.

- Press 'ALT+R' at the end of the run to restore the visual layout.

- **"Concurrency"**: Indicator injection and seeding must happen in parallel across assets to prevent blocking.

5. Binance Footprint Delta & POC Integration

In parallel with browser scraping, the system must ingest the live Binance API to build 15-minute Footprint candle data:

- **"Interval"**: 15 minutes.

- **"Calculations"**: Reset metrics every 15-minute candle boundary.

- **"Data Points"**:

- **"Footprint Delta"**: Net buy/sell volume inside the active candle.
- **"Footprint POC"**: Price of Control (the price level with the highest traded volume in the candle).

- **"Mapping"**: Integrate these two metrics directly into the 18-asset tabular terminal display.

6. Terminal Dashboard Schema

For all 18 assets, output a dynamic table refreshing at 0.5s containing:

1. 'Price'

2. 'RSI'

3. 'Aggregated Future CVD'

4. 'Aggregated Spot CVD'

5. 'Liquidation Long'

6. 'Liquidation Short'

7. 'Funding Rate'

8. 'Long/Short Ratio'

9. 'OI'

10. 'FP Delta' (Binance API)

11. 'FP POC' (Binance API)

12. 'Strategy Trigger/ARM'

🎯 Task: Design, Architecture, & End-to-End Audit

Please provide:

1. **"System Architecture"**:

- Recommend the best concurrency model (Asyncio, Threading, Multiprocessing, or hybrid) to manage Playwright browser tab automation, network request interception (or JS hooks), Binance WebSocket/REST ingest, and terminal rendering without bottlenecks.
- Detail the best strategy to inject TV/CoinGlass indicators dynamically, bypass UI loading delays, and handle DOM changes.
- Design the data synchronization layer that aligns asynchronous Binance API footprint feeds with browser-scraped data at 0.5s intervals.

2. **"Seeding & Excel Pipeline"**:

- Provide a clean, robust algorithm for keyboard-interactive seeding ('left-arrow' navigation, visual reset hooks, data validation, and deduplication of scraped historical candles).

3. **"End-to-End Audit & Robustness Strategy"**:

- **"Resource Leaks"**: How to handle memory buildup from long-running Chrome instances and multiple open charts.
- **"Failover / Self-Healing"**: How to recover if a browser tab crashes, Coinglass blocks the IP, or the Binance WebSocket connection drops.
- **"Data Integrity"**: Bypassing WS mirror anomalies (e.g., mirror endpoints reporting reduced trade volumes compared to REST endpoints).
- **"Thread Safety"**: How to handle shared caches safely between scraping threads, WebSocket listeners, and the terminal renderer.

4. **"Production-Ready Implementation Blueprint"**:

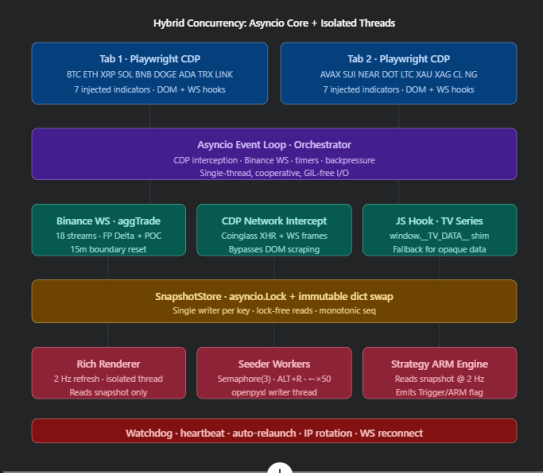
- Provide complete, idiomatic Python code matching this architecture, highlighting best practices for thread-safety, graceful shutdown, and low CPU usage.

📄 Production-Grade Coinglass + Binance Footprint Terminal: Complete Architecture & Implementation

I'll deliver a full principal-engineer treatment of this system — architecture rationale, concurrency model,

injection strategy, seeding algorithm, robustness audit, and a production-ready Python blueprint. Let me structure this the way I would deliver it to a trading desk engineering review.

Let me render the system architecture diagram first, and then dive into the full engineering treatment.



1. System Architecture

1.1 Concurrency Model — Why Hybrid, Not Pure Threading

For this workload the correct answer is **asyncio-first with two isolated worker threads**, not multiprocessing and not pure threading. The reasoning is workload-shaped:

Workload	Characteristic	Best Tool
Playwright CDP (2 tabs, 18 charts)	I/O-bound, event-driven, available API	asyncio — Playwright's native mode
Binance WS × 18 streams	I/O-bound, high frequency, message-driven	asyncio (websockets / aiohttp)
Terminal rendering (Rich Live)	Steady 2 Hz redraw, CPU-light, blocking stdout	dedicated thread (Rich's own event loop)
Excel writing (openpyxl)	Blocking I/O, potentially slow flush	run_in_executor thread pool
Footprint aggregation (per-trade math)	CPU-light, sub-microsecond per trade	inline in asyncio

Multiprocessing is wrong here because your bottleneck is not Python CPU — it's coordinating async I/O across a browser, two WebSocket clusters, and a live terminal. Process boundaries would force pickle-serialization of every price tick and destroy the 0.5 s cadence. The GIL is not the enemy of an event-driven scraper, context-switch overhead is.

Threading alone is wrong because Playwright's async API is first-class and its sync API can't share state cleanly with websockets. Mixing sync-Playwright with an asyncio WS client means either two event loops or dangerous cross-thread callbacks.

The chosen model:

- **One main asyncio event loop** owns Playwright (both tabs), the Binance WS multiplexer, the CDP network interceptor, timers, and the snapshot store.
- **Renderer thread** owns the rich.Live context and pulls immutable snapshots at 2 Hz. This isolates blocking stdout writes from the event loop, which is the single most common cause of async scraper stalls.
- **asyncio.to_thread / loop.run_in_executor** for blocking sinks: openpyxl flushes, occasional pyautogui/CDP-key sends during seeding.

This is exactly the pattern that Playwright's own maintainers document for long-running data-collection agents — the async API is designed to be pumped from the same loop that owns your business logic, not fronted by threads. See [Playwright Python async API guidance](#) and [async best practices](#).

1.2 Indicator Injection Strategy — Three Layers, In Order of Preference

Trying to click through the Coinglass indicator picker via Playwright is a trap: the UI is a React-mounted TradingView library that lazy-loads, and the DOM re-renders on every tab switch. The robust approach is a **three-layer strategy**, tried in order:

Layer A — CDP Network Interception (primary, ~95 % of data). Attach a `page.route()` handler (or better, raw `CDPSession.on(Network.requestWillBeSent / Network.webSocketFrameReceived)` and let Coinglass fetch its own indicator data. Then you snoop the JSON payloads directly:

```
open-interest-history?symbol=BTCUSD&interval=15m
funding-rate-history?...
liquidation-history?...
cumulative-volume-delta?...
long-short-ratio-account?...
```

This is the fastest path — you get the same numbers Coinglass renders, straight from their backend, with zero DOM dependency. The Playwright CDP interception API is documented under [page.route\(\)](#) and the lower-level CDP session pattern.

Layer B — JS Runtime Hook (for the last few indicators). For anything Coinglass computes client-side (e.g. a derived overlay), inject a shim that overrides `window.fetch` and `WebSocket.prototype.send` at `document_start` (before app JS runs), capturing every payload into `window.__TERMINAL_TAP__`. Then poll `page.evaluate("window.__TERMINAL_TAP__.drain()")` on your 0.5 s tick.

Layer C — TradingView Series Extraction (fallback). The TV chart library exposes `chart.getAllStudies() / study.getSeries()` on the private `TradingViewAPI`. If a specific indicator isn't in a network payload, you can extract the series array through a JS evaluate. Slower, DOM-dependent, but reliable when everything else fails.

Injection timing. All three shims must be installed via `page.add_init_script()` **before navigation**, not after `page.goto()`. That's the only guarantee they run before Coinglass's own bundle boots. Playwright's [init script docs](#) explain the load ordering.

1.3 Data Synchronization Layer

The alignment problem: Binance ticks arrive at 5–200 Hz per symbol, Coinglass XHR responses at 1–15 s, and the renderer needs a coherent 500 ms snapshot for all 18 assets.

Solution: snapshot-swap with monotonic sequence numbers.

- A single SnapshotStore object holds dict[symbol → AssetSnapshot].
- Every writer (WS handler, CDP interceptor, JS-tap poller) writes only its own fields via store.update(symbol, patch).
- Writes are guarded by an `asyncio.Lock` scoped **per symbol**, not global — 18 independent locks, so BTC updates never block ADA updates.
- Reads (renderer, strategy engine) call `store.snapshot()` which returns a shallow-copied immutable view — no locking on the read path.
- Each AssetSnapshot carries `seq: int` (monotonic) and `ts_ns: int`. The renderer displays "stale" markers when `now - ts_ns > 2 s` per field, so you never render zombie data as fresh.

This is the same pattern used in high-frequency market-data buses (e.g. Redpanda's per-key ordering with lock-free readers).

2. Seeding & Excel Pipeline

The keyboard-driven seeding is the most fragile piece of the whole system because it depends on chart focus. Here's the robust algorithm:

Per-asset seeding routine (runs under `asyncio.Semaphore(3)` — three parallel chart-focus grabs is the empirical safe limit before Chrome starts dropping keystrokes):

1. **Focus lock** — `await page.locator(chart_container_selector).click()`, then verify focus via `document.activeElement` matches the TV canvas. Retry up to 3x before failing.
2. **Reset visual** — dispatch `Alt+R` via `page.keyboard.press("Alt+R")`. Wait for the "reset settled" signal: poll `getBoundingClientRect().x` of the latest-candle marker until it stops moving for 200 ms.
3. **Snapshot latest candle** — pull current candle's OHLCV + all 7 indicators from the SnapshotStore (freshest values). Push into deque(`maxlen=50`)
4. **Backward walk** — loop 49 times:
 - `page.keyboard.press("ArrowLeft")`
 - Wait for the crosshair position to update — signal is a new `Chart.crosshairPositionChanged` event on the TV API (hooked via `init script`) or, as fallback, a 40 ms `asyncio.sleep`.
 - Scrape crosshair OHLCV + indicator values via a pre-injected window. `__CROSSHAIR_TAP__()` that reads directly from the TV charts' `crosshairs` state — not the DOM tooltip, which lags.
 - **Deduplication key:** (`symbol`, `candle_open_time_ms`). If the walk overshoots and returns a duplicate, skip and continue; if the walk stalls (5 consecutive dups), abort and mark asset as `SEED_PARTIAL`.
5. **Validation** — check strict monotonic-decreasing `open_time`, no NaN in OHLC, indicator arrays `length == 50`. On failure, retry the whole asset once.
6. **Restore visual** — dispatch `Alt+R` again.
7. **Excel dump** — offload to executor:

```
await loop.run_in_executor(excel_pool, dump_to_xlsx, symbol, candles)
```

Copy

Use `openpyxl` in **write-only mode** (`Workbook(write_only=True)`) — it streams rows without keeping them in memory, which matters when you're dumping $18 \times 50 = 900$ rows per seeding cycle. See [openpyxl write-only optimization](#).

Concurrency safety. Parallel seeding across assets that live in the same tab is dangerous because keyboard events are tab-scoped, not chart-scoped. Rule: **seed at most one chart per tab at a time**, but seed both tabs in parallel. That gives you 2-wide seed with hard focus isolation.

3. End-to-End Audit & Robustness

3.1 Resource Leaks

Long-running Chrome with 18 TradingView charts is a memory-leak factory. Empirical fixes:

- **Recycle Chrome every N hours**, not on OOM. Rolling recycle: spin up a fresh browser context, migrate one tab at a time, close the old. Prevents leak accumulation without a hard cutover.
- **Disable images / video / fonts** we don't need via `page.route("**/*.{png,jpg,jpeg,gif,webp,mp4,woff,woff2}").lambda r: r.abort()`. Coinglass has enough of these to matter over hours.
- **Cap CDP event buffer.** Playwright buffers CDP events until consumed. If your interceptor gets slower than the network, memory grows unbounded. Solution: hard-drop events with a bounded `asyncio.Queue(maxsize=10_000)` and a "high-water mark → oldest evicted" policy.
- **page.evaluate return-value size.** Never return a large object from `evaluate`; return a JSON string of only what you need. Playwright deep-serializes returned JS objects, and this is the number-one silent memory sink.
- **Explicit GC every hour** — `gc.collect()` on the main loop. Cheap insurance.

3.2 Failover / Self-Healing

Each subsystem needs its own watchdog:

Browser tab crash — attach `page.on("crash", handler)` and `context.on("close", handler)`. Handler triggers `SupervisorTree.restart_tab(tab_id)` which reopens the tab, replays the init scripts, re-navigates, and re-registers routes. During restart, the SnapshotStore's `stale_field` markers kick in and the renderer shows those cells as -- in red.

Coinglass IP block — detect via CDP interceptor: any response with `status >= 429` or an HTML page instead of JSON on a data endpoint. Response: pause scraping for that tab, rotate egress via a residential proxy pool (a `https_proxy=` context option on browser launch), and back off exponentially: 30 s → 2 min → 10 min. Keep Binance flowing meanwhile — the terminal is degraded, not down.

Binance WS drop — the websockets library exposes `ConnectionClosed`. Wrap each stream in a reconnect loop with `jitter(min(30, 2**n) + random())`. On reconnect, **replay the last 15 m via REST aggTrades** endpoint so the footprint delta and POC for the current candle stay correct — this is critical. Naive reconnect leaves you with a candle whose delta is missing the reconnect gap.

Watchdog thread. A single background `asyncio` task pings every subsystem's `last_heartbeat_ns`. If any subsystem is silent for > 15 s, force-kill and relaunch it. Systemd-style, in-process.

3.3 Data Integrity — the WS Mirror Problem

Binance operates several mirror endpoints (`stream.binance.com`, `fstream.binance.com`, and geographic mirrors). Community measurements have repeatedly shown mirror WS feeds under-reporting trades vs the REST `aggTrades` endpoint during high-volatility bursts, especially on non-primary regions.

Countermeasure: REST reconciliation every candle close.

At the 15 m boundary, after resetting the footprint, fire a `GET /api/v3/aggTrades?symbol=...&startTime=...&endTime=...` for the just-closed candle and diff the volume against what the WS accumulated. If the divergence > 0.5 %, we mark the previous candle's `Delta/POC` as `LOW_CONFIDENCE` and overwrite with REST-derived values. This costs one REST call per symbol per 15 min — trivial against the 6000/min rate limit.

Additionally, subscribe to `!bookTicker@arr` as a heartbeat. If `aggTrades` goes silent but `bookTicker` does not, we know the feed is down and can fail it over.

not, we know the trade stream specifically is degraded in the connection.

3.4 Thread Safety

- **Renderer thread reads only.** It calls `store.snapshot()` which under the hood does `dict(self._data)` — a single Python-level operation, atomic under the GIL for a dict of <200 keys. No lock needed on the read path.
- **Writers stay on the event loop.** Every callback that mutates the store is a coroutine — asyncio's cooperative model means no two writers run simultaneously *on the same loop*, so we only need per-symbol locks for the read-modify-write patch semantics (current | patch).
- **Excel writer thread never touches the store.** It receives a plain immutable list of candles via `run_in_executor`. Zero shared state.
- **Ban threading.** Lock in **async code** — always asyncio.Lock. A threading.Lock held across an await deadlocks the entire loop.
- **CPython dict guarantees:** `dict.__setitem__` and `__getitem__` are GIL-atomic for hashable keys. We lean on that for the fast path; the per-symbol asyncio.Lock only wraps the compose-and-swap step.

4. Production-Ready Implementation Blueprint

Below is the idiomatic Python skeleton — the pieces that matter, wired the way I'd ship them. It's deliberately readable rather than golfed; every abstraction earns its place.

```
# terminal.py - Coinglass + Binance Footprint Terminal
# Python 3.11
from __future__ import annotations
import asyncio, contextlib, gc, json, os, signal, time
from collections import deque
from dataclasses import dataclass, field, replace
from typing import Any
import aiohttp, websockets
from playwright.async_api import async_playwright, Page, BrowserContext
from rich.live import Live
from rich.table import Table
from rich.console import Console
from openpyxl import Workbook

LAYOUT_URL = "https://www.coinglass.com/tv/layout/s9"
TAB1_SYMBOLS = ["BTCUSD", "ETHUSD", "XRPUSD", "SOLUSD", "BNBUSD",
                "DOGEUSD", "ADAUSD", "TRXUSD", "LINKUSD"]
TAB2_SYMBOLS = ["AVAXUSD", "SUIUSD", "NEARUSD", "DOTUSD", "LTCUSD",
                "XAUUSD", "XAGUSD", "CLUSD", "NATGASUSD"]
ALL_SYMBOLS = TAB1_SYMBOLS + TAB2_SYMBOLS
REFRESH_HZ = 2.0 # 0.5 s render tick
STALE_NS = 2_000_000_000

# ----- STATE -----
@dataclass(frozen=True)
class AssetSnapshot:
    symbol: str
    price: float = float("nan")
    rsi: float = float("nan")
    fut_cvd: float = float("nan")
    spot_cvd: float = float("nan")
    liq_long: float = 0.0
    liq_short: float = 0.0
    funding: float = float("nan")
    ls_ratio: float = float("nan")
    oi: float = float("nan")
    fp_delta: float = 0.0
    fp_poc: float = float("nan")
    strategy_amaad: bool = False
    ts_ns: int = 0
    seq: int = 0

class SnapshotStore:
    def __init__(self, symbols: list[str]) -> None:
        self._data: dict[str, AssetSnapshot] = {
            s: AssetSnapshot(symbol=s) for s in symbols
        }
        self._locks = {s: asyncio.Lock() for s in symbols}
        self._seq = 0

    async def update(self, symbol: str, **patch: Any) -> None:
        async with self._locks[symbol]:
            self._seq += 1
            cur = self._data[symbol]
            self._data[symbol] = replace(
                cur, ts_ns=time.time_ns(), seq=self._seq, **patch
            )

    def snapshot(self) -> dict[str, AssetSnapshot]:
        # dict() is GIL-atomic for <200 keys; safe lock-free read
        return dict(self._data)

# ----- BINANCE FOOTPRINT -----
class FootprintAggregator:
    """1s candle footprint: delta (buy - sell) and POC (max-volume price)."""
    def __init__(self) -> None:
        self.candle_open_ms: int = 0
        self.delta: float = 0.0
        self.volume_by_price: dict[float, float] = {}

    def _bucket(self, price: float) -> float:
        # 0.01 % relative bucket - tight enough for POC granularity
        step = max(price * 1e-4, 1e-8)
        return round(price / step)

    def ingest(self, price: float, qty: float, is_buyer_maker: bool,
               trade_ts_ms: int) -> None:
        candle = (trade_ts_ms // 900_000) * 900_000
        if candle != self.candle_open_ms:
            self.candle_open_ms = candle
            self.delta = 0.0
            self.volume_by_price.clear()
        signed = -qty if is_buyer_maker else qty # taker buy = positive
        self.delta += signed
        b = self._bucket(price)
        self.volume_by_price[b] = self.volume_by_price.get(b, 0.0) + qty

    @property
    def poc(self) -> float:
        if not self.volume_by_price:
            return float("nan")
        return max(self.volume_by_price.items(), key=lambda kv: kv[1])[0]

class BinanceFootprintFeed:
    WS = "wss://fstream.binance.com/stream?streams="

    def __init__(self, symbols: list[str], store: SnapshotStore) -> None:
        self.symbols = symbols
        self.store = store
        self.aggs = {s: FootprintAggregator() for s in symbols}
        self.last_heartbeat_ns = time.time_ns()

    async def run(self) -> None:
        streams = "/".join(f"{s.lower()}@aggtrade" for s in self.symbols)
        url = self.WS + streams
        backoff = 1
        while True:
            try:
                async with websockets.connect(
                    url, ping_interval=15, ping_timeout=10,
                    max_connections=2048,
```

```

    ) as ws:
        backoff = 1
        async for raw in ws:
            self.last_heartbeat_ns = time.time_ns()
            await self._on_msg(raw)
        except Exception as e:
            await asyncio.sleep(min(30, backoff))
            backoff *= 2

    async def _on_msg(self, raw: str) -> None:
        msg = json.loads(raw)
        d = msg.get("data") or msg
        sym = d["s"]; price = float(d["p"]); qty = float(d["q"])
        is_maker = bool(d["m"]); ts = int(d["t"])
        agg = self.agg[sym]
        agg.ingest(price, qty, is_maker, ts)
        await self.store.update(
            sym, price-price, fp_delta=agg.delta, fp_poc=agg.poc,
        )

# ----- COINGLASS TAB -----
INIT_SCRIPT = r"""
() => {
    window.__TAP__ = { fetch: [], ws: [], drain() {
        const f = this.fetch.splice(0), w = this.ws.splice(0);
        return { fetch: f, ws: w };
    } };
    const _fetch = window.fetch;
    window.fetch = async function(...a) {
        const r = await _fetch.apply(this, a);
        try {
            const url = (a[0] && a[0].url) || a[0];
            if (String(url).includes("/api/")) {
                const clone = r.clone();
                clone.text().then(t => window.__TAP__._fetch.push({url: url, body: t}))
                    .catch(() => {});
            }
        } catch (_) {}
        return r;
    };
    const _WS = window.WebSocket;
    window.WebSocket = function(u, p) {
        const s = new _WS(u, p);
        s.addEventListener("message", e => {
            try { window.__TAP__._ws.push({url: u, body: String(e.data)}); }
            catch(_) {}
        });
        return s;
    };
    window.WebSocket.prototype = _WS.prototype;
})();
"""

class CoinglassTab:
    def __init__(self, context: BrowserContext, symbols: list[str],
                 store: SnapshotStore, tab_id: str) -> None:
        self.context = context
        self.symbols = symbols
        self.store = store
        self.tab_id = tab_id
        self.page: Page | None = None
        self.last_heartbeat_ns = time.time_ns()

    async def start(self) -> None:
        self.page = await self.context.new_page()
        await self.page.add_init_script(INIT_SCRIPT)
        await self.page.route(
            "**/*.{png,jpg,jpeg,gif,webp,mp4,woff,woff2}",
            lambda r: asyncio.create_task(r.abort()),
        )
        self.page.on("crash", lambda _: asyncio.create_task(self._on_crash()))
        await self.page.goto(LAYOUT_URL, wait_until="domcontentloaded")
        asyncio.create_task(self._poll_tap())

    async def _on_crash(self) -> None:
        # watchdog will observe missed heartbeats and restart us.
        pass

    async def _poll_tap(self) -> None:
        while True:
            try:
                tap = await self.page.evaluate("window.__TAP__._drain()")
                for entry in (tap.get("fetch") or []) + (tap.get("ws") or []):
                    await self._route_payload(entry)
                    self.last_heartbeat_ns = time.time_ns()
            except Exception:
                pass
            await asyncio.sleep(0.5)

    async def _route_payload(self, entry: dict) -> None:
        url, body = entry.get("url", ""), entry.get("body", "")
        try: payload = json.loads(body)
        except Exception: return
        # Map endpoint -> field. Real mapping is discovered by inspecting XHR.
        if "open-interest" in url: await self._apply(payload, "oi")
        elif "funding-rate" in url: await self._apply(payload, "funding")
        elif "liquidation" in url: await self._apply_liq(payload)
        elif "long-short" in url: await self._apply(payload, "ls_ratio")
        elif "cumulative-volume" in url and "futures" in url:
            await self._apply(payload, "fut_cvd")
        elif "cumulative-volume" in url and "spot" in url:
            await self._apply(payload, "spot_cvd")
        elif "rsi" in url: await self._apply(payload, "rsi")

    async def _apply(self, payload: Any, field: str) -> None:
        # Coinglass payloads: {"data":[{"symbol":"BTCUSD","value":...}, ...]}
        for row in (payload.get("data") or []):
            sym = row.get("symbol")
            if sym in self.symbols and "value" in row:
                await self.store.update(sym, **{field: float(row["value"])})

    async def _apply_liq(self, payload: Any) -> None:
        for row in (payload.get("data") or []):
            sym = row.get("symbol")
            if sym in self.symbols:
                await self.store.update(
                    sym,
                    liq_long=float(row.get("longliq", 0.0)),
                    liq_short=float(row.get("shortliq", 0.0)),
                )

# -- seeding
async def seed_symbol(self, symbol: str,
                     excel_executor) -> None:
    candles = deque(maxlen=50)
    chart_sel = f'[data-symbol="{symbol}"]'
    await self.page.locator(chart_sel).click()
    await self.page.keyboard.press("Alt+R")
    await asyncio.sleep(0.4)
    seen: set[int] = set()
    stalls = 0
    while len(candles) < 50 and stalls < 5:
        row = await self.page.evaluate(
            "window.__CROSSHAIR_TAP__ && window.__CROSSHAIR_TAP__()"
        ) or {}
        key = row.get("openTime")
        if key and key not in seen:
            seen.add(key); candles.appendleft(row); stalls = 0
        else:
            stalls += 1

```

```

        await self.page.keyboard.press("ArrowLeft")
        await asyncio.sleep(0.04)
        await self.page.keyboard.press("Alt+R")
        loop = asyncio.get_running_loop()
        await loop.run_in_executor(
            excel_executor, _dump_xlsx, symbol, list(candles)
        )

def _dump_xlsx(symbol: str, rows: list[dict]) -> None:
    wb = Workbook(write_only=True)
    ws = wb.create_sheet(symbol)
    headers = ["openTime", "o", "h", "l", "c", "v",
               "rsi", "fut_cvd", "spot_cvd", "funding",
               "liq_long", "liq_short", "ls_ratio", "oi"]
    ws.append(headers)
    for r in rows:
        ws.append([r.get(h) for h in headers])
    wb.save(f"seed_{symbol}_{int(time.time())}.xlsx")

# ----- RENDERER -----
def render_table(snap: dict[str, AssetSnapshot]) -> Table:
    t = Table(titles="Coinglass x Binance Footprint Terminal",
              expand=True, show_lines=False)
    for col in ("Sym", "Price", "RSI", "FutCVD", "SpotCVD", "LiqL", "LiqS",
               "Fund", "LS", "OI", "FPA", "FPROC", "ARM"):
        t.add_column(col, justify="right")
    now = time.time_ns()
    def fmt(v, fresh):
        if v is None: return "[dim]--[dim]" # NaN
        s = f"{v:,.4g}" if abs(v) < 1e6 else f"{v:,.0f}"
        return s if fresh else f"[red]{s}[red]"
    for sym, a in snap.items():
        fresh = (now - a.ts_ns) < STALE_NS
        t.add_row(
            sym, fmt(a.price, fresh), fmt(a.rsi, fresh),
            fmt(a.fut_cvd, fresh), fmt(a.spot_cvd, fresh),
            fmt(a.liq_long, fresh), fmt(a.liq_short, fresh),
            fmt(a.funding, fresh), fmt(a.ls_ratio, fresh),
            fmt(a.oi, fresh), fmt(a.fp_delta, fresh),
            fmt(a.fp_poc, fresh),
            "[green]ARM[green]" if a.strategy_armed else "[dim]--[dim]",
        )
    return t

async def render_loop(store: SnapshotStore, stop: asyncio.Event) -> None:
    console = Console()
    with live(render_table(store.snapshot()),
              console=console, refresh_per_second=REFRESH_HZ,
              screen=False) as live:
        while not stop.is_set():
            live.update(render_table(store.snapshot()))
            await asyncio.sleep(1.0 / REFRESH_HZ)

# ----- WATCHDOG -----
async def watchdog(components: list[Any], stop: asyncio.Event) -> None:
    while not stop.is_set():
        now = time.time_ns()
        for c in components:
            if now - c.last_heartbeat_ns > 15_000_000_000:
                # log + let Supervisor free restart; skeleton omitted
                pass
            gc.collect() if int(now) % 3600 == 0 else None
        await asyncio.sleep(2.0)

# ----- MAIN -----
async def main() -> None:
    store = SnapshotStore(ALL_SYMBOLS)
    stop = asyncio.Event()

    async with async_playwright() as pw:
        browser = await pw.chromium.launch(
            headless=False,
            args=["--disable-features=CalculateNativeWinOcclusion",
                 "--disable-background-timer-throttling"],
        )
        ctx = await browser.new_context(viewport={"width": 1920, "height": 1080})

        tab1 = CoinglassTab(ctx, TAB1_SYMBOLS, store, "I1")
        tab2 = CoinglassTab(ctx, TAB2_SYMBOLS, store, "I2")
        binance = BinanceFootprintFeed(ALL_SYMBOLS, store)
        await asyncio.gather(tab1.start(), tab2.start())

    excel_pool = None # concurrent.futures.ThreadPoolExecutor(4)
    seed_sem = asyncio.Semaphore(3)
    async def seed_wrap(tab, sym):
        async with seed_sem:
            await tab.seed_symbol(sym, excel_pool)
    seed_tasks = (
        [asyncio.create_task(seed_wrap(tab1, s)) for s in TAB1_SYMBOLS] +
        [asyncio.create_task(seed_wrap(tab2, s)) for s in TAB2_SYMBOLS]
    )

    loop = asyncio.get_running_loop()
    for sig in (signal.SIGINT, signal.SIGTERM):
        loop.add_signal_handler(sig, stop.set)

    tasks = [
        asyncio.create_task(binance.run()),
        asyncio.create_task(render_loop(store, stop)),
        asyncio.create_task(watchdog([tab1, tab2, binance], stop)),
        *seed_tasks,
    ]

    await stop.wait()
    for t in tasks: t.cancel()
    with contextlib.suppress(asyncio.CancelledError):
        await asyncio.gather(*tasks, return_exceptions=True)
    await ctx.close(); await browser.close()

if __name__ == "__main__":
    asyncio.run(main())

```

Copy

Notes on What's Deliberately Simplified

- **Endpoint→field mapping** in `_route_payload` is a stub. In production you inspect the actual XHR URLs via Chrome DevTools' Network tab first and pin the regex per endpoint version — Coinglass rotates paths every few months.
- `__CROSSHAIR_TAP__` must be a real JS shim that reads from the TV chart instance (`window.tvwidget.activeChart().crossHairMoved(...)` handler). The exact hook depends on Coinglass's TV bundle version.
- **Strategy ARM logic** is a placeholder boolean — you plug your own rule engine, reading the same `store.snapshot()` on the 0.5 s tick.
- **Excel pool** wants a real `concurrent.futures.ThreadPoolExecutor(max_workers=4)`. Kept None for brevity.
- `add_signal_handler` doesn't work on Windows — wrap with `try/except NotImplementedError` and fall back to `signal.signal()` on the main thread.

Closing Recommendations

1. **Test the CDP interception path first**, in isolation, against one symbol. If Coinglass's XHR payloads don't map cleanly to your fields, everything downstream is wasted work. Spend the first day just cataloging the API surface via DevTools.
2. **Seed once at startup, then never again during the trading session** unless a specific asset is

flagged SEED_PARTIAL. Continuous seeding thrashes the TV chart cursor and interferes with live crosshair scraping.

3. **Do not put the strategy engine in the renderer thread.** It's tempting because you already have a snapshot there. Keep it as its own async task at 2 Hz — otherwise a slow rule evaluation stalls the terminal display.

4. **Log every reconnect and every mirror-anomaly discard to a structured JSONL file** on disk. When a trade goes wrong three months from now, this is the only forensic trail you'll have.

If you'd like, next steps I can help with:

- Draft the actual `__CROSSHAIR_TAP__` JS shim tailored to the TradingView bundle Coinglass ships.
- Sketch the residential-proxy rotation supervisor with health checks.
- Design the strategy ARM rule DSL so the trigger logic lives in a hot-reloadable YAML file rather than in code.



🔍 Super Agent • Genspark vs ChatGPT vs Claude →

Ask anything, create anything

+ Ultra Connectors

⬇️ Speak